



Figure 4: **Camera-conditioned text-to-video generation.** Comparison of the proposed approach to MotionCtrl and CameraCtrl for the same camera trajectory input. We adapted MotionCtrl and CameraCtrl to the same transformer-based model for fair comparisons. MotionCtrl exhibits worse quality due to fine-tuning existing layers, and CameraCtrl is not sensitive to camera conditioning. See the supplementary webpage for video results.

self-attention layers, which are the core computational component of FIT. It is crucial to instantiate the weights of the output convolutions $\text{Conv}_{\text{res}}^{(b)}$ from zeros to preserve the model initialization. Besides, we initialize the weights of $\text{FF}_{\text{cam}}^{(b)}$ and $\text{XAttn}_{\text{cam}}^{(b)}$ from the corresponding parameters of the original network. This approach helps to preserve visual quality at initialization and facilitates rapid fine-tuning on a small dataset. As a result, we obtain the method for fine-grained 3D camera control in large video diffusion transformers. We name it VD3D and visualize its architecture in Figure 3.

3.3 TRAINING DETAILS

To ensure comparability with prior work such as (Wang et al., 2023e), we train our video generator on the same RealEstate10K dataset (Zhou et al., 2018). We optimize only the newly added parameters $\text{FF}_{\tilde{p}}$ and $(\text{FF}_{\text{cam}}^{(b)}, \text{XAttn}_{\text{cam}}^{(b)})_{b=1}^B$, and keep the rest of the network frozen. We found that training only the base 36×64 model was sufficient, as the 288×512 upsampler already accurately follows the camera motion of a low-resolution video.

3.4 DATASET

We fine-tune a text-to-video model, pre-trained on 2D video data, on RealEstate10K (Zhou et al., 2018). The training split for fine-tuning consists of roughly 65K video clips, and is the same as is used in concurrent work (MotionCtrl (Wang et al., 2023e) and CameraCtrl (He et al., 2024a)).

3.5 METRICS

We conduct a user study to evaluate our approach for camera-controlled text-to-video generation. The study participants are presented with 20 side-by-side comparisons between the proposed approach and the baselines as well as a reference video from RealEstate10K with the same trajectory to better judge the camera alignment. We ask 20 participants for each generated video sequence to indicate which generated video they prefer based on multiple submetrics, namely, camera alignment, motion quality, text alignment, visual quality, and overall preference. The user study involved negligible risk to the participants and was conducted with appropriate institutional review board and legal approval. We evaluate our method using 20 camera trajectories sampled from the RealEstate10K test split that were not seen during training for the user study. We use the full test split combined with unseen text prompts for the automated camera evaluations, i.e., 6928 unseen camera trajectories combined with out-of-distribution text prompts.

3.6 BASELINES

We compare our work to MotionCtrl (Wang et al., 2023e) and the concurrent work CameraCtrl (He et al., 2024a) by adapting their publicly released code to the same pre-trained video model as ours. Note that both baselines were originally designed for space-time disentangled U-Net video diffusion models. Hence, their approaches are not directly applicable to the spatio-temporal transformers, and so we adapt them to this setting as follows. For MotionCtrl, we omit their object motion control module and use their proposed camera motion control module to encode the camera parameters into a context vector used with the SnapVideo model. We fine-tune both the camera motion control module and the cross attention between the latent vectors and the patches. For CameraCtrl, we fine-tune the original camera encoder module and use this to produce the latent vectors in the SnapVideo model. During fine-tuning the model weights are kept frozen—i.e., the same as in our proposed approach. Furthermore, we include comparisons to the original MotionCtrl and CameraCtrl works built on the U-Net-based AnimateDiff (Guo et al., 2024) architecture. For fair comparison, we evaluate all metrics for MotionCtrl (U-Net) and CameraCtrl (U-Net) on the same test sets as our method using the publicly provided code and checkpoints. Moreover, we provide metrics for the base model used across all our experiments, i.e., a model without camera injection. We provide more baseline variants of MotionCtrl and CameraCtrl in the appendix in Sec. B.3.

4 EXPERIMENTS

4.1 ASSESSMENT

We provide a qualitative and quantitative assessment of our approach compared to the baselines in Fig. 4 and in Tab. 1. Following CameraCtrl (He et al., 2024a), we also evaluate the camera pose accuracy using ParticleSfM (Zhao et al., 2022) on generated videos in Tab. 2. We use generations for text prompts from RealEstate10K (Zhou et al., 2018) and MSR-VTT (Xu et al., 2016), testing both in- and out-of-distribution prompts. Note that we adjust the CameraCtrl (He et al., 2024a) evaluation pipeline by normalizing all cameras into a unified scale as COLMAP provides different scales across different scenes. This prevents scenes with large scale to have a higher impact on the errors. Please also refer to the supplementary webpage for additional video results.

In Fig. 4 we observe that adapting the camera conditioning method from the MotionCtrl degrades visual quality and text alignment, likely because this approach adjusts the weights of the base video model. In the space-time U-Net for which this approach was proposed, the temporal layers can be fine-tuned without sacrificing visual fidelity. Since spatio-temporal transformers do not decompose temporal and spatial attributes in the same way, the model overfits to the small dataset used to fine-tune the cross-attention layer. While we observe some agreement with the camera poses used to condition the model, the text alignment is generally low in our experiments (see supplemental webpage). In contrast, CameraCtrl keeps the pre-trained video model weights frozen and only trains a camera encoder. This leads to strong visual quality, but the generated videos show little agreement with the input camera poses. For fair comparison, we trained all models for the same number of iterations (described in Sec. 3.3).

The results of the user study in Tab. 1 show that most participants prefer the generated videos using the proposed camera conditioning mechanism across all evaluated sub-metrics. We also observe a pronounced preference for the camera alignment of the proposed method compared to the other baselines. That is, 82% and 78% of participants prefer the camera alignment of the proposed method compared to our respective adaptations of MotionCtrl and CameraCtrl to the video transformer model. All results are significant at the $p < 0.001$ level as evaluated using a χ^2 test. We further present image-based metrics for multi-view generation and video generation quality metrics in the appendix in Sec. B.1 and Tab. B.2 respectively. The results of our camera pose accuracy evaluation in Tab. 2 further demonstrate that our approach clearly outperforms previous works. Note that the previous works MotionCtrl and CameraCtrl especially struggle with the rotation accuracy.

4.2 ABLATIONS

Plücker embedding. We motivate our Plücker embedding conditioning mechanism by training a variant using the raw camera matrices. Concretely, we flatten and concatenate extrinsics and intrinsics

Table 1: **Quantitative results.** We compare our method to MotionCtrl and CameraCtrl implemented on the same base video models as ours. The methods are evaluated in a user study in which participants indicate their preference based on camera alignment (CA), motion quality (MQ), text alignment (TA), visual quality (VQ), and overall preference (Overall). The percentages indicate preference for VD3D vs. the alternative method (in each row). All results are statistically significant with $p < 0.001$ as evaluated using a χ^2 test.

Method	Human Preference				
	CA	MQ	TA	VQ	Overall
VD3D vs. MotionCtrl	82%	81%	86%	81%	84%
VD3D vs. CameraCtrl	78%	64%	63%	65%	66%

Table 2: **Camera pose evaluation.** We evaluate all models using reference camera trajectories from the RealEstate10K test set. We compute translation and rotation errors based on estimated camera poses from generations using ParticleSfM (Zhao et al., 2022).

Method	RealEstate10K		MSR-VTT	
	TransError ($\downarrow$)	RotError ($\downarrow$)	TransError ($\downarrow$)	RotError ($\downarrow$)
Base Model	0.616	0.207	0.717	0.216
MotionCtrl (U-Net)	0.477	0.094	0.593	0.137
CameraCtrl (U-Net)	0.465	0.089	0.587	0.132
MotionCtrl	0.518	0.161	0.627	0.148
CameraCtrl	0.532	0.165	0.578	0.220
Ours	0.409	0.043	0.504	0.050
w/o Plucker	0.517	0.161	0.676	0.156
w/o ControlNet	0.573	0.182	0.787	0.179
w/o weight copy	0.424	0.044	0.513	0.063
w/o add context	0.602	0.212	0.702	0.128
w/o Plucker context	0.487	0.088	0.627	0.091

matrices in the channel dimension and repeat the values in the spatial patch dimensions. We observe that Plucker embeddings provide an essential spatial conditioning mechanism, as shown in Tab. 2.

ControlNet conditioning. Our ControlNet-inspired conditioning mechanism ensures fast and precise learning of the conditioning signal distribution. Instead of using a ControlNet block, we simply add zero-initialized Plucker embedding features to the patches and observe close to no camera control. We observe training cross-attention layers in the ControlNet block is key to learning camera control while preserving the original model weights. This is confirmed by our camera evaluation in Tab. 2.

ControlNet weight copy. While the ControlNet block is essential, copying the pre-trained weights into the new copy has rather minor impact, as shown in Tab. 2. To verify this, we train a model where we randomly initialize the cross-attention block between patches and latents instead of copying the weights. We observe similar results, showing that rather the architecture and zero-initialization are the key component of this design.

Add to context vector. We train a model where we use camera matrices, linearly encode them, and add them to the context vector as a simple conditioning mechanism for transformer-based models. The rendered videos show little to no correlation with the input camera matrices, resulting in poor camera pose accuracy, as shown in Tab. 2. This highlights the importance of carefully incorporating cameras into the spatio-temporal patches with our Plucker embedding block.

Plucker in context vector. We train a model where we integrate Plucker embeddings into the context vector instead of the patches. We similarly compute Plucker embeddings, but instead of patchifying them, we flatten and linearly map the embeddings into a vector that has the same shape as the context vector. We fuse the Plucker features and context vector using the same Plucker conditioning mechanism used for the patches. We observe higher camera pose errors in Tab. 2, highlighting that it